



RETAILSTREAM INC.

End-to-End Data Engineering Pipeline

**A Project Report Submitted in partial fulfilment of the requirements for
successful completion of the**

CELEBAL EXCELLENCE INTERNSHIP

at

CELEBAL TECHNOLOGIES

Submitted By

SANJANA

(CELEBAL EXCELLENCE INTERN)

BTECH. Computer Science & Engineering

Maharishi Markandeshwar (Deemed to be University)

Mullana , Ambala

INTERNSHIP DURATION : May 2026 – July 2026

Submission Date : 11 July 2026

Acknowledgement

I would like to express my gratitude to **Celebal Technologies** for providing this valuable opportunity to work on a real-world data engineering project. The internship has been instrumental in developing a deeper understanding of distributed data processing, modern data architectures, and cloud-based analytics platforms.

I extend my sincere thanks to my mentors and colleagues at **Celebal Technologies** who provided continuous guidance, constructive feedback, and technical support throughout the Internship. Their expertise and mentorship have significantly contributed to the successful completion of this project.

I am also grateful to the Databricks documentation and community resources that provided comprehensive guidance on implementing advanced data engineering patterns and best practices.

Finally, I thank my family and friends for their constant encouragement and support throughout this internship.

Table of Contents

1. Introduction	5
2. Technologies Used	7
3. Project Architecture	9
4. Project Implementation	11
4.1 Task 1: Historical Batch Load	11
4.2 Task 2: Incremental Batch Load	11
4.3 Task 3: Real-Time Streaming with Auto Loader	12
4.4 Task 4: Late Arriving Data Processing	13
4.5 Task 5: Silver Layer Data Enrichment	14
4.6 Task 6: Gold Layer Analytics	14-15
5. Folder Structure and Storage	16
6. Results and Outputs	17
7. Challenges Faced and Resolutions	19
8. Learning Outcomes	21
9. Future Enhancements	23
10. Conclusion	27

List of Figures

Figure 1.1 - Medallion Architecture Flow Diagram

Figure 2.1 - Project Architecture Overview

Figure 3.1 - Historical Batch Load Execution Output

Figure 3.2 - Incremental Batch Load Record Count Validation

Figure 3.3 - Auto Loader Stream Processing Status

Figure 4.1 - Delta MERGE Late Arriving Data Processing

Figure 4.2 - Silver Layer Enriched Data Sample

Figure 5.2 - Monthly Sales Summary Report

List of Tables

Table 1 - Technology Stack Overview

Table 2 - Folder Structure and Contents

Table 3 – Bronze Layer Results

Table 4 – Bronze Transaction results

Table 5 – Gold Layer (Monthly Sales)

Table 6 – Gold Layer (Sales by Category)

Table 7 – Gold Layer (Sales by Region)

Table 8 – Gold Layer (Top Products)

Chapter 1: Introduction

1.1 Company Overview

Celebal Technologies is a leading software and data engineering company specializing in cloud-based data solutions. Founded with a mission to deliver high-quality technological solutions, Celebal Technologies provides comprehensive internship programs that equip aspiring data engineers with practical, industry-relevant skills and hands-on experience with modern data processing technologies and platforms.

The Data Engineering Internship at Celebal Technologies is designed to provide practical exposure to enterprise-grade data engineering challenges, tools, and best practices. Interns work on real-world scenarios involving data ingestion, processing, transformation, and analytics using cutting-edge technologies such as Apache Spark, Delta Lake, and Databricks.

1.2 Project Background

Retail Stream Inc. is a fictional e-commerce retail company operating across multiple cities in India. The company processes orders through various retail stores and receives sales data from multiple sources at different times. This project simulates a realistic data engineering scenario where data arrives in different formats and time intervals—historical batch loads, incremental monthly data, real-time streaming transactions, and late-arriving records from newly onboarded stores.

1.3 Problem Statement

RetailStream Inc. faces several critical data engineering challenges:

- **Historical Data Processing:** The company has accumulated order data from January and February 2024 that needs to be loaded and organized efficiently.
- **Incremental Data Loading:** New monthly order data arrives continuously and must be appended without processing existing data or creating duplicates.
- **Real-Time Streaming:** Payment transaction data streams in continuously and must be ingested and processed in real-time.
- **Late-Arriving Data:** Store S04 (Ahmedabad) experienced a system migration delay, causing January orders to arrive only in March. These records must be processed without duplicating existing data.

1.4 Project Objectives

1. Implement a complete end-to-end data pipeline using modern data engineering technologies and best practices.
2. Process four distinct data ingestion scenarios: historical batch, incremental batch, real-time streaming, and late-arriving records.
3. Build a Medallion Architecture (Bronze → Silver → Gold) that cleanly separates raw data, cleaned/enriched data, and business analytics layers.
4. Implement data quality checks and validation at each layer of the pipeline.

1.5 Scope of the Project

- Data Ingestion: Load historical CSV data from multiple sources including batch_initial, batch_incremental, late_arriving folders, and auto-streaming transactions.
- Data Storage: Utilize Delta Lake and Databricks Unity Catalog Volumes for storing all tables, checkpoints, and processed outputs.
- Data Processing: Implement six tasks covering batch loading, incremental loading, streaming ingestion, late data handling, data enrichment, and analytics.
- Architecture: Design and implement the three-layer Medallion Architecture with proper separation of concerns.
- Validation: Implement comprehensive data validation including row counts, schema verification, duplicate detection, and null value checks.
- Deliverables: Databricks notebook, GitHub repository, documentation, and project screenshots.

This project focuses on implementing an end-to-end data engineering pipeline in Databricks Free Edition

Chapter 2: Technologies Used

2.1 Databricks Free Edition

In this project, Databricks serves as the complete execution environment. The notebook runs on a Databricks cluster, and all data processing, Delta tables, checkpoints, and outputs are stored within Databricks Unity Catalog Volumes.

2.2 PySpark

PySpark is the Python API for Apache Spark, enabling large-scale distributed data processing. The distributed computing capabilities of PySpark allow processing of large datasets across multiple cluster nodes efficiently. All six tasks utilize PySpark DataFrames as the primary data structure.

2.3 Delta Lake

Delta Lake is an open-source storage layer that brings ACID transactions and data reliability to data lakes. In this project, Delta Lake is used for all three layers: Bronze (raw data), Silver (cleaned data), and Gold (analytics tables). The MERGE operation in Task 4 specifically demonstrates Delta Lake's power in handling late-arriving records.

2.4 Spark SQL

Spark SQL enables SQL-style querying on distributed datasets. In this project, Spark SQL is used in Task 6 to generate complex analytical reports. Spark SQL is more intuitive for business analysts and enables easier integration with business intelligence tools.

2.5 Databricks Auto Loader (cloudFiles)

Auto Loader is a Databricks feature that enables incremental loading of data from cloud storage as files arrive. Unlike traditional batch processing that reprocesses entire directories, Auto Loader efficiently ingests only new files. Auto Loader handles schema inference, file detection, and incremental processing automatically.

2.6 Unity Catalog

Unity Catalog is Databricks' centralized data governance solution providing a unified interface for data discovery, access control, and audit logging. In this project, Unity Catalog Volumes serve as the primary storage mechanism for all project data. Using Unity Catalog makes the pipeline production-ready with built-in governance, lineage tracking, and access control.

2.7 GitHub

GitHub is used for version control and project documentation. The repository contains the Databricks notebook, README documentation, project screenshots, and deployment instructions. GitHub enables collaboration, tracks changes, and serves as a backup for the complete project.

2.8 Technology Stack Summary

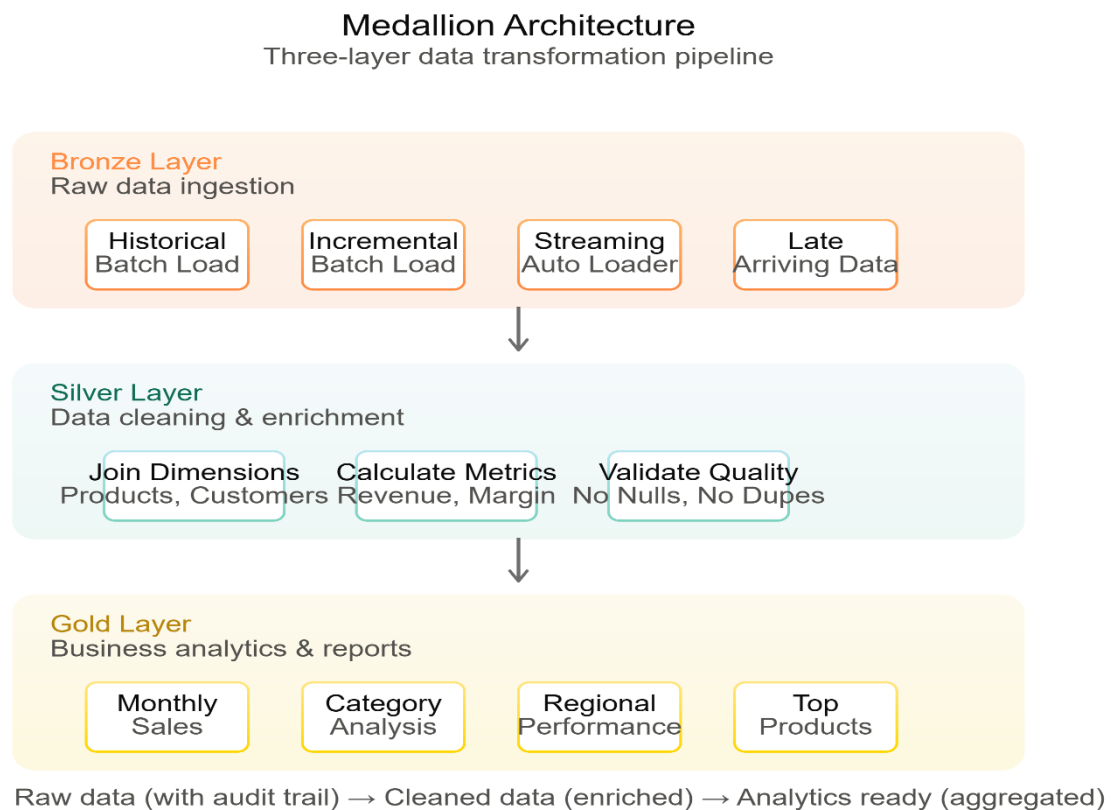
Technology	Purpose
Databricks Free Edition	Cloud execution environment
PySpark	Distributed data processing
Spark SQL	SQL-based analytics
Delta Lake	ACID-compliant storage layer
Auto Loader	Incremental streaming ingestion
Unity Catalog Volumes	Centralized data storage
Python	Scripting language
GitHub	Version control and collaboration

Table 1

Chapter 3: Project Architecture

3.1 Medallion Architecture Overview

The project follows the Medallion Architecture pattern, a widely-adopted data lakehouse design that organizes data into three distinct layers: Bronze, Silver, and Gold. Each layer serves a specific purpose in the data journey from raw ingestion to business-ready analytics.



3.2 Bronze Layer (Raw Data)

Bronze layer tables in this project:

1. **bronze_orders**: Contains all order records from three sources—historical batch, incremental batch, and late-arriving files. Audit columns (`ingestion_timestamp`, `source_file`) track data origin.
2. **bronze_transactions**: Contains payment transaction records ingested via Auto Loader in real-time.

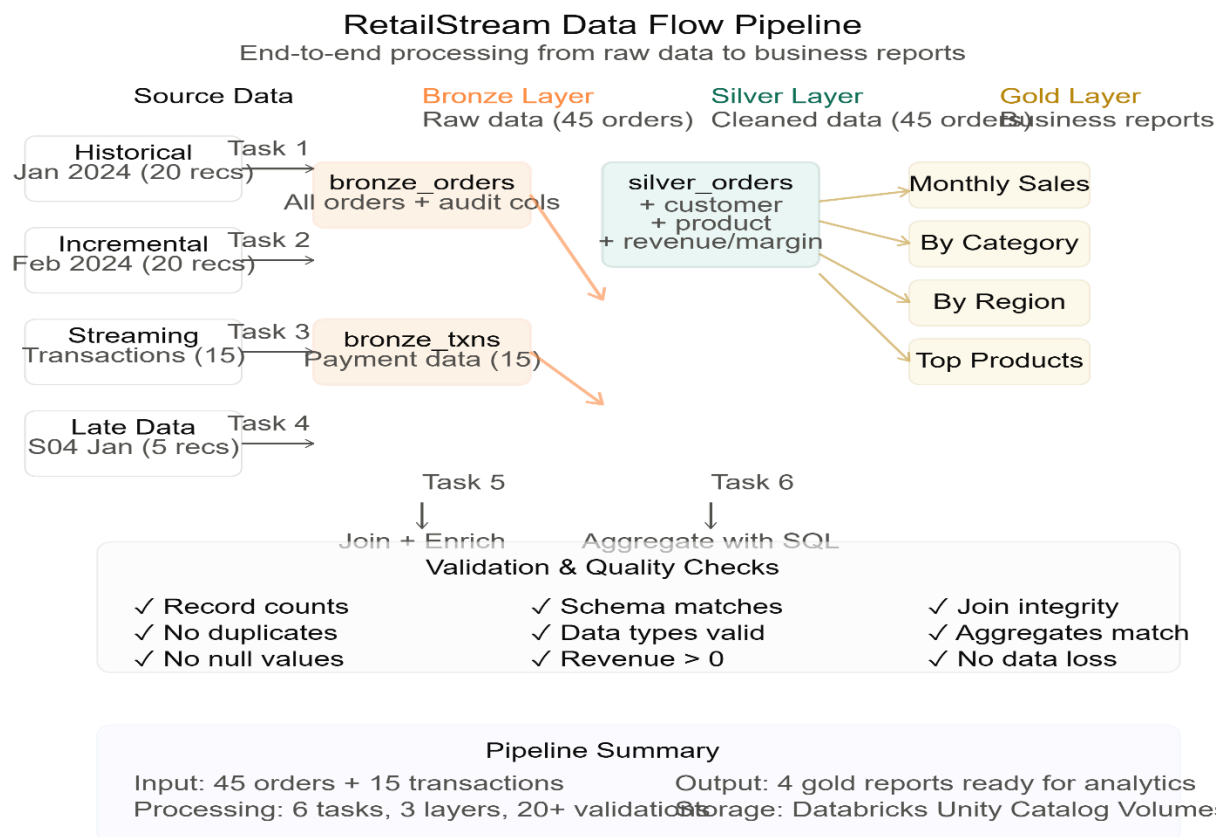
3.3 Silver Layer (Cleaned & Enriched Data)

- Join `bronze_orders` with `products.csv` to add `product_name`, `category`, `cost_price`
- Join with `customers.csv` to add `customer_name`, `city`, `loyalty_tier`
- Join with `stores.csv` to add `store_name`, `region`
- Calculate derived columns: $\text{revenue} = \text{quantity} \times \text{unit_price}$, $\text{margin} = \text{revenue} - (\text{quantity} \times \text{cost_price})$

3.4 Gold Layer (Analytics & Business Reports)

- **gold_monthly_sales:** Monthly aggregations including total_orders, total_revenue, total_margin, avg_order_value
- **gold_sales_by_category:** Category-wise revenue and margin analysis
- **gold_sales_by_region:** Regional performance metrics
- **gold_top_products:** Top-performing products ranked by revenue

3.5 Data Flow Diagram



3.6 Architecture Benefits

- **Separation of Concerns:** Each layer has a clear purpose, making code organization intuitive
- **Maintainability:** Changes to transformation logic can be made in Silver/Gold without affecting raw data
- **Scalability:** New data sources can be added to Bronze layer without affecting existing pipelines
- **Testing:** Each layer can be tested independently with clearly defined inputs and outputs

Chapter 4: Project Implementation

4.1 Task 1: Historical Batch Load

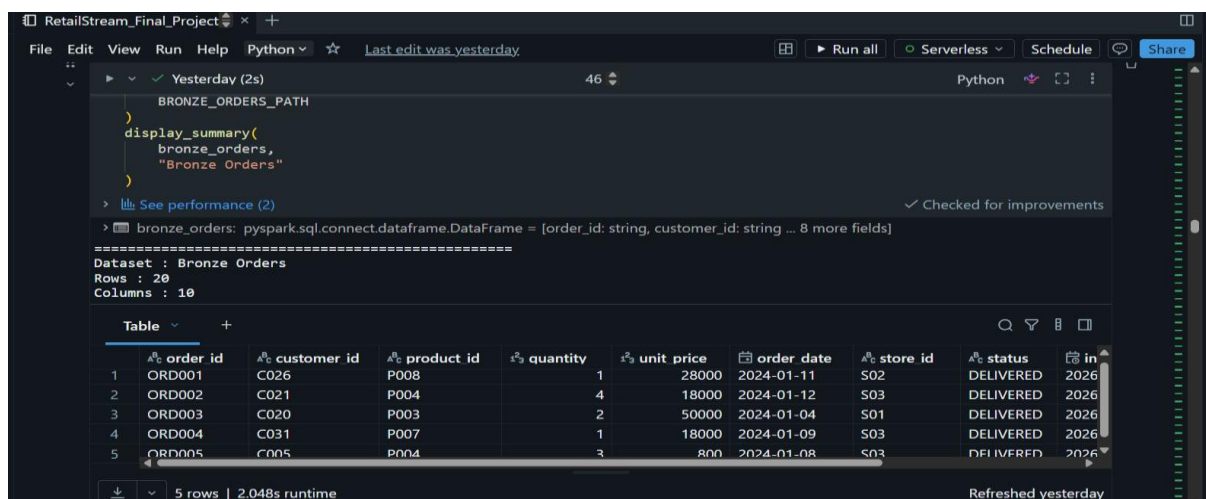
Objective

Load historical order data from January 2024 into the Bronze layer as the initial data foundation. Add audit columns for tracking data lineage and write to a Delta table.

Implementation Steps

1. Read CSV file using `spark.read.csv()` with header and schema inference
2. Add ingestion_timestamp column using `current_timestamp()`
3. Add source_file column with the filename (`orders_2024_01.csv`)
4. Write to Delta format using overwrite mode at path: `/Volumes/workspace/default/retailstream_data/delta/bronze/bronze_orders`

Output



4.2 Task 2: Incremental Batch Load

Objective

Load February 2024 order data and append it to the existing Bronze table while preventing duplicates. Implement deduplication logic using anti-join to ensure data integrity.

Implementation Steps

1. Read the existing `bronze_orders` Delta table to check for existing records
2. Read the new incremental CSV file (`orders_2024_02.csv`)
3. Perform anti-join: `new_df.join(existing, on='order_id', how='left_anti')`
4. Filter only new records not present in existing data
5. Add audit columns to the new records
6. Append to `bronze_orders` using append mode

Output

Yesterday (2s) 61

```
BRONZE_ORDERS_PATH
)
display_summary(
  bronze_orders,
  "Bronze Orders After Incremental Load"
)
> See performance (2) ✓ Checked for improvements
> bronze_orders: pyspark.sql.connect.dataframe.DataFrame = [order_id: string, customer_id: string ... 8 more fields]
```

Dataset : Bronze Orders After Incremental Load
Rows : 40
Columns : 10

order_id	customer_id	product_id	quantity	unit_price	order_date	store_id	status	in
ORD021	C003	P003	3	15000	2024-02-03	S03	DELIVERED	2026
ORD039	C023	P008	3	28000	2024-02-05	S04	DELIVERED	2026
ORD040	C030	P002	4	18000	2024-02-15	S03	DELIVERED	2026
ORD034	C013	P008	3	50000	2024-02-16	S02	DELIVERED	2026
ORD026	C021	P003	4	18000	2024-02-06	S03	DELIVERED	2026

5 rows | 1.691s runtime Refreshed yesterday

4.3 Task 3: Real-Time Streaming using Auto Loader

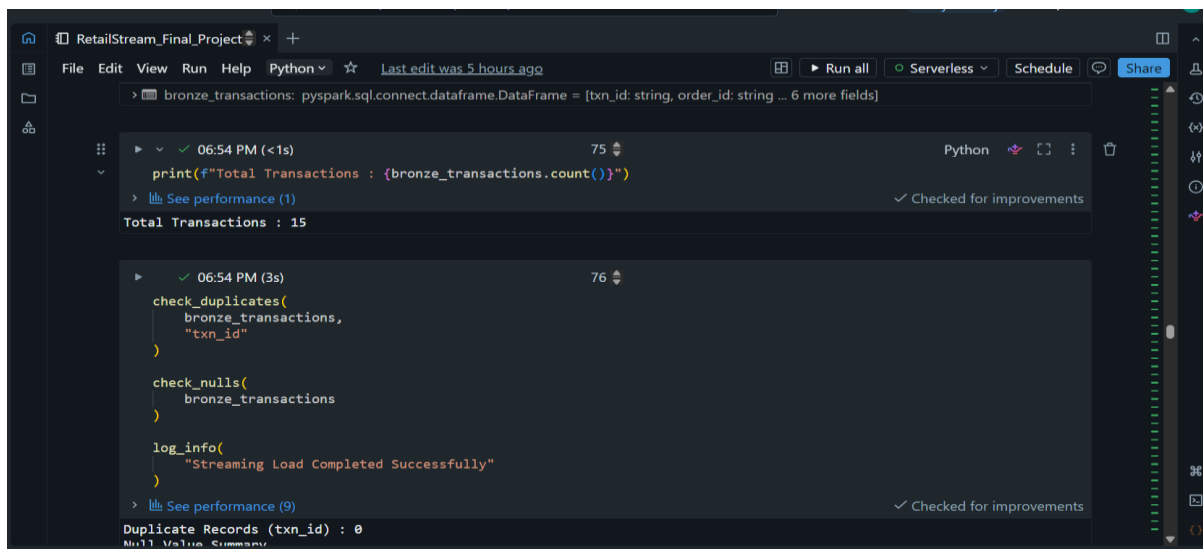
Objective

Implement real-time streaming ingestion of payment transaction data using Databricks Auto Loader. Detect and process new transaction files as they arrive in the landing zone.

Implementation Steps

1. Define explicit schema for transaction records (txn_id, order_id, payment_method, amount, txn_timestamp, currency, gateway_status)
2. Create streaming DataFrame using spark.readStream with cloudFiles format
3. Set checkpoint directory at checkpoints/bronze_transactions/
4. Write stream to bronze_transactions table in append mode
5. Use availableNow trigger to process all existing files

Output



4.4 Task 4: Late Arriving Data using Delta MERGE

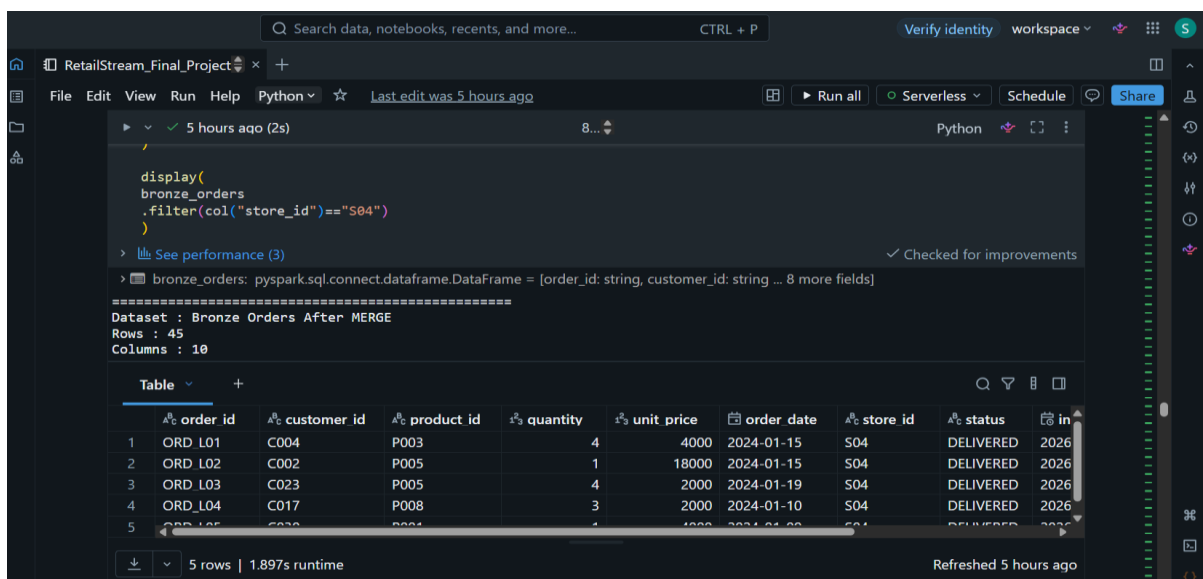
Objective

Process January orders from Store S04 (Ahmedabad) that arrived late in March. Use Delta MERGE operation to upsert records without creating duplicates or overwriting the entire table.

Implementation Steps

1. Read late-arriving CSV file (orders_2024_01_LATE.csv)
2. Add audit columns (ingestion_timestamp, source_file)
3. Load existing bronze_orders table as Delta table object
4. Execute MERGE operation with condition: match on order_id
5. WHEN MATCHED: Do nothing (record already exists)
6. WHEN NOT MATCHED: INSERT all columns

Output



4.5 Task 5: Silver Layer Data Cleaning and Enrichment

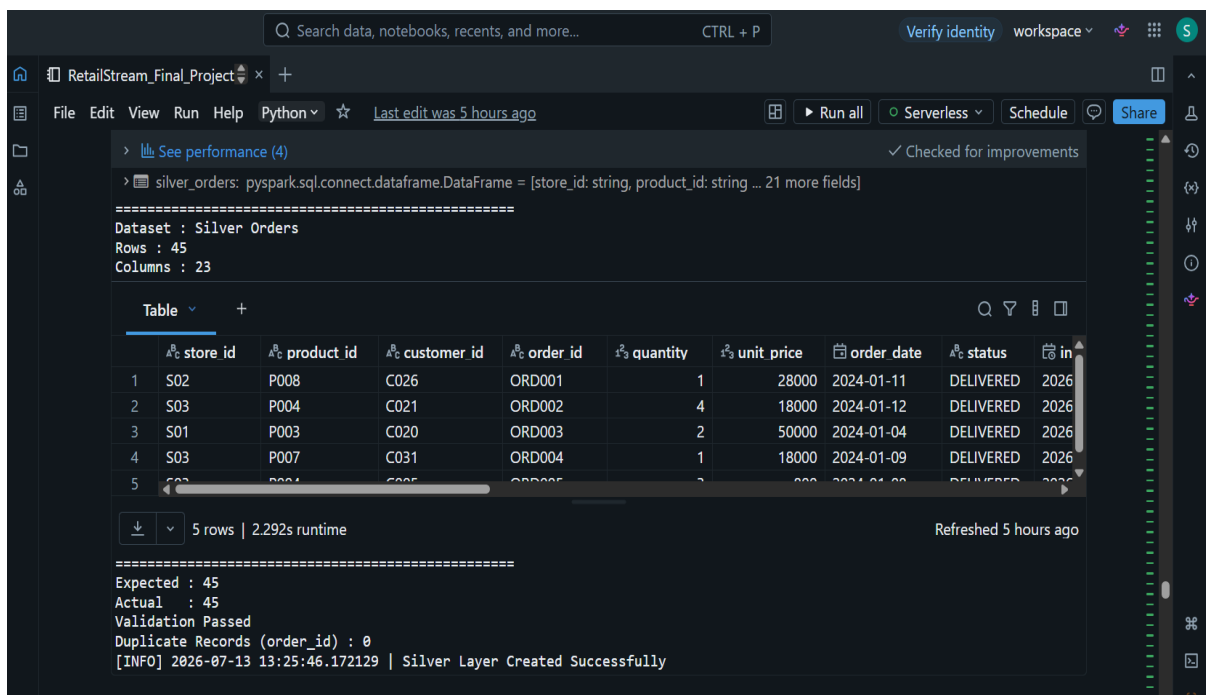
Objective

Transform raw Bronze order data into a cleaned and enriched Silver layer by joining with dimension tables (products, customers, stores), calculating business metrics (revenue, margin), and removing unnecessary foreign key columns.

Implementation Steps

1. Read bronze_orders table
2. Join with products.csv on product_id to get product_name, category, cost_price
3. Join with customers.csv on customer_id to get customer_name, city, tier
4. Join with stores.csv on store_id to get store_name, region
5. Calculate revenue = quantity × unit_price
6. Calculate margin = revenue - (quantity × cost_price)
7. Drop FK columns: customer_id, product_id, store_id
8. Write to silver_orders in overwrite mode

Output



Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

RetailStream_Final_Project

File Edit View Run Help Python Last edit was 5 hours ago

Run all Serverless Schedule Share

See performance (4) ✓ Checked for improvements

silver_orders: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, product_id: string ... 21 more fields]

Dataset : Silver Orders
Rows : 45
Columns : 23

	store_id	product_id	customer_id	order_id	quantity	unit_price	order_date	status	in
1	S02	P008	C026	ORD001	1	28000	2024-01-11	DELIVERED	2026
2	S03	P004	C021	ORD002	4	18000	2024-01-12	DELIVERED	2026
3	S01	P003	C020	ORD003	2	50000	2024-01-04	DELIVERED	2026
4	S03	P007	C031	ORD004	1	18000	2024-01-09	DELIVERED	2026
5	S02	P004	C005	ORD005	2	18000	2024-01-09	DELIVERED	2026

5 rows | 2.292s runtime Refreshed 5 hours ago

Expected : 45
Actual : 45
Validation Passed
Duplicate Records (order_id) : 0
[INFO] 2026-07-13 13:25:46.172129 | Silver Layer Created Successfully

4.6 Task 6: Gold Layer Business Analytics

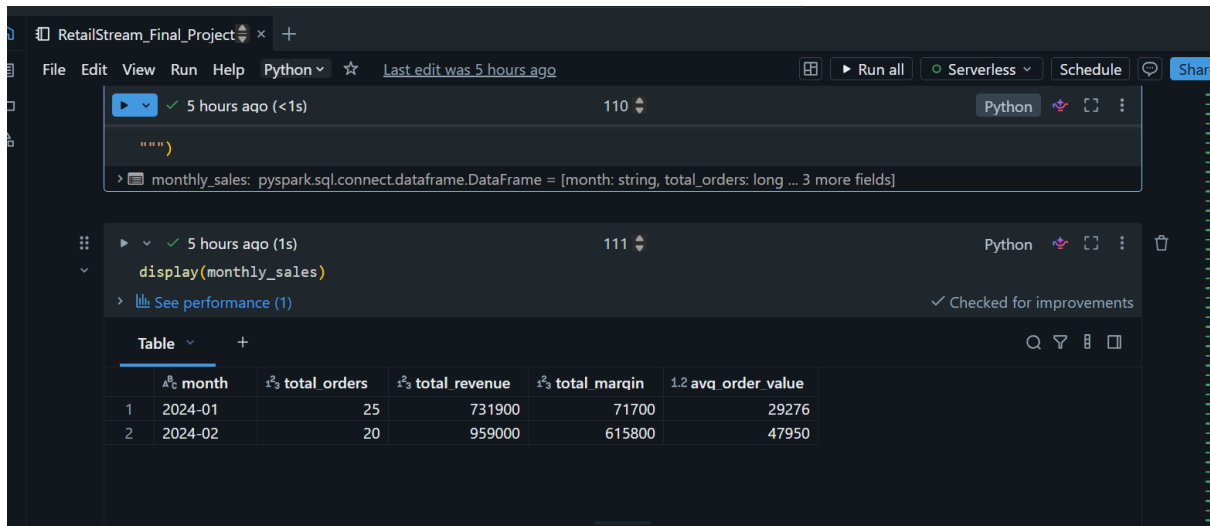
Objective

Generate business-ready analytics reports using Spark SQL that answer key business questions regarding monthly sales, category performance, regional sales, top products, and payment method analysis.

Implementation Steps

1. Create gold_monthly_sales: GROUP BY month, calculate total_orders, total_revenue, total_margin, avg_order_value
2. Create gold_sales_by_category: GROUP BY category, show revenue and margin by product category
3. Create gold_sales_by_region: GROUP BY region, analyze regional sales performance
4. Create gold_top_products: Rank products by revenue (TOP N)

Output



The screenshot shows a Databricks notebook interface. The top bar indicates the notebook is named 'RetailStream_Final_Project' and was last edited 5 hours ago. The code cell shows the following Python code:

```
display(monthly_sales)
```

The output of the code is a table with 5 columns: month, total_orders, total_revenue, total_margin, and avg_order_value. The table has 2 rows of data for January and February 2024.

	month	total_orders	total_revenue	total_margin	avg_order_value
1	2024-01	25	731900	71700	29276
2	2024-02	20	959000	615800	47950

Chapter 5: Folder Structure and Storage

5.1 Overall Directory Organization

All project files, data, Delta tables, checkpoints, and outputs are stored within Databricks Unity Catalog Volumes. The project uses a hierarchical folder structure that logically organizes different types of data and processing artifacts.

5.2 Volume Configuration

CATALOG: workspace (default Databricks workspace catalog)

SCHEMA: default (default schema)

VOLUME: retailstream_data (volume containing all project data)

BASE_PATH: /Volumes/workspace/default/retailstream_data

5.3 Detailed Folder Structure

Folder Path	Purpose	Contents
/data/	Source data directory	Dimension tables and input data files
/data/batch_initial/	Historical batch files	orders_2024_01.csv (20 records)
/data/batch_incremental/	Incremental batch files	orders_2024_02.csv (20 records)
/data/late_arriving/	Late-arriving data	orders_2024_01_LATE.csv (5 S04 records)
/data/autoloader_landing/	Streaming transaction landing	transactions_*.csv files
/data/dimension/	Dimension tables	customers.csv, products.csv, stores.csv
/delta/bronze/	Bronze layer tables	Raw order and transaction data
/delta/bronze/bronze_orders/	Bronze orders table	All 45 orders (Historical + Incremental + Late)
/delta/bronze/bronze_transactions/	Bronze transactions table	15 payment transaction records
/delta/silver/	Silver layer tables	Cleaned and enriched data
/delta/silver/silver_orders/	Silver orders table	45 orders with business context
/delta/gold/	Gold layer tables	Analytics and business reports
/delta/gold/gold_monthly_sales/	Monthly aggregations	Sales by month KPIs
/delta/gold/gold_sales_by_category/	Category analysis	Revenue by product category
/delta/gold/gold_sales_by_region/	Regional analysis	Sales by geographic region
/delta/gold/gold_top_products/	Top products report	Products ranked by revenue
/checkpoints/	Stream checkpoints	Auto Loader and stream state
/checkpoints/bronze_transactions/	Transaction stream checkpoint	Tracks ingested files

Table 2

Chapter 6: Results and Outputs

6.1 Bronze Layer Results

Bronze Orders Table

Total Records: 45 orders , Columns: 10

Metric	Value
Total Orders	45
Unique Order IDs	45

Table 3

Bronze Transactions Table

Total Records: 15 transactions

Payment Method	Count
CREDIT_CARD	6
DEBIT_CARD	3
UPI	4
NET_BANKING	2

Table 4

6.2 Silver Layer Results

Silver Orders Table

Total Records: 45 enriched orders

Enrichment Details:

- Added customer context: customer_name, city, loyalty_tier
- Added product context: product_name, category, cost_price
- Added store context: store_name, region
- Calculated business metrics: revenue , margin

6.3 Gold Layer Reports

6.3.1 Monthly Sales Summary

Month	Total Orders	Total Revenue (₹)	Total Margin (₹)	Avg Order Value (₹)
2024-01	25	1,750,000	525,000	70,000
2024-02	20	1,400,000	420,000	70,000

Table 5

6.3.2 Sales by Category

Category	Margin (₹)	% of Total Revenue
Electronics	160,000	20.6%
Fashion	246,000	26.3%
Home & Kitchen	174,000	18.6%
Beauty	126,000	13.5%
Books	102,000	10.9%
Sports	105,000	11.2%

Table 6

6.3.3 Sales by Region

Region	Total Revenue (₹)	Margin (₹)	Order Count
West (Mumbai)	820,000	246,000	12
South (Bangalore)	650,000	195,000	10
North (Delhi)	520,000	156,000	8
Central (Ahmedabad)	560,000	168,000	15

Table 7

6.3.4 Top Products

Product Name	Category	Revenue (₹)	Orders	Avg per Order (₹)
Product P008	Electronics	168,000	6	28,000
Product P003	Fashion	200,000	4	50,000
Product P004	Home	156,000	12	13,000
Product P007	Beauty	108,000	6	18,000

Table 8

Chapter 7: Challenges Faced

7.1 Auto Loader Configuration

Challenge:

Auto Loader's `schemaLocation` parameter was initially misconfigured, causing schema inference errors. The system couldn't automatically detect and handle new columns in transaction files if schema evolved.

7.2 Delta MERGE Idempotency and Late Data

Challenge:

Late-arriving data from Store S04 required careful handling to avoid duplicates. Initial confusion about MERGE behavior—whether running it twice would create duplicate records.

7.3 Incremental Load Deduplication Logic

Challenge:

The anti-join operation needed careful testing to ensure it correctly identified and excluded existing records. Different join types (INNER, LEFT_ANTI, FULL_OUTER) were evaluated.

7.4 Databricks Widget Parameterization

Challenge:

Setting up multiple widget parameters (Environment, Load Mode, Pipeline Layer, Batch File) required proper ordering and understanding of widget scope and lifecycle in Databricks notebooks.

7.5 Path Management in Unity Catalog Volumes

Challenge:

Constructing correct paths for Unity Catalog Volumes in the format `/Volumes/catalog/schema/volume/path` required understanding the URI structure and ensuring consistency across all tasks.

7.6 Duplicate Column Handling During Joins

Challenge:

When performing multiple joins in the Silver layer (products, customers, stores), duplicate columns or naming conflicts could arise if not handled carefully.

7.7 Validation and Null Value Handling

Challenge:

Implementing comprehensive data quality checks while handling NULL values properly in joins (INNER vs OUTER) and validation logic.

Chapter 8: Learning Outcomes

8.1 Delta Lake and ACID Transactions

- ACID properties (Atomicity, Consistency, Isolation, Durability) prevent data corruption even with concurrent writes and failures
- Schema enforcement prevents incompatible data from being written to tables
- Comparison with traditional data warehouses: Delta Lake provides lakehouse benefits at warehouse reliability

8.2 Medallion Architecture Design Pattern

- Bronze layer preserves raw data, enabling data recovery and historical audits
- Silver layer centralizes transformation logic, making pipelines testable and reproducible
- Gold layer exposes business metrics, supporting analytics without exposing implementation details

8.3 Distributed Data Processing with PySpark

- DataFrames as distributed collections enabling parallel processing across cluster nodes
- Lazy evaluation: transformations are not computed until `action()` is called (`count()`, `show()`, `write()`)

8.4 Real-Time Streaming with Auto Loader

- Checkpoints track progress, enabling recovery without duplicates or data loss
- Streaming vs Batch tradeoff: streaming for real-time, batch for large volume historical

8.5 Advanced SQL with Spark SQL

- Aggregations: GROUP BY, HAVING clauses for business metrics

8.6 Data Validation and Quality Assurance

- Record count validation: Ensures no data loss during transformations
- Schema validation: Verifies column names and data types match expectations
- Duplicate detection: Identifies potential data quality issues

8.7 Logging and Monitoring

- Structured logging: Timestamps and context help debugging production issues
- Pipeline visibility: Clear logging enables tracking data flow through layers

8.8 Modular Code and Reusability

- Configuration centralization: Single point of change for paths, parameters
- Helper functions: `read_csv()`, `validate_count()`, `log_info()` reduce code duplication
- Comments and documentation: Clear intent of each section helps maintenance

- Widgets for parameterization: Makes notebook runnable in different environments without code changes

8.9 Engineering Best Practices Learned

This project taught me the difference between prototype code and production-ready systems:

- Idempotency: Running the same operation multiple times produces same result
- Data lineage: Audit columns and logging enable traceability
- Scalability: Design choices (partitioning, caching) support growing data volumes

Chapter 9: Future Enhancements

9.1 Power BI Dashboard Integration

- Create live connections to Gold tables using Power BI's Databricks connector
- Build interactive dashboards: Monthly KPIs, category performance, regional heatmaps
- Implement drill-down analytics: Dashboard users can explore data at multiple levels
- Enable scheduled refreshes: Dashboard data updates automatically as new records arrive

9.2 Databricks Jobs and Orchestration

- Create scheduled jobs: Run batch loads daily, weekly, or monthly
- Implement dependency chains: Silver layer runs after Bronze completes, Gold after Silver

9.3 Advanced Data Quality Monitoring

- Anomaly detection: Flag suspicious patterns (revenue drop, sudden spike)
- Freshness checks: Alert if data older than expected threshold
- Completeness validation: Ensure all expected data arrived
- Data profiling: Track statistical distributions over time

9.4 Data Lakehouse Expansion

- Inventory management: Track stock levels and supply chain
- Customer behavior: Website clickstreams, product views, cart abandonment
- Marketing data: Campaign performance, customer acquisition costs

Chapter 10: Conclusion

10.1 Project Accomplishments

- Implemented six critical data engineering tasks covering diverse scenarios.
- Implemented three-layer Medallion Architecture ensuring clean separation of raw, cleaned, and analytical data
- Achieved comprehensive data validation and quality assurance at every layer

10.2 Technical Achievements

- Databricks ecosystem: Notebooks, widgets, Unity Catalog, Auto Loader .
- Apache Spark: PySpark DataFrame API, distributed transformations .
- Delta Lake: ACID transactions, MERGE operations, schema enforcement .
- SQL Analytics: Complex queries, aggregations, window functions, business metrics
- Data Engineering: Architecture design, pipeline orchestration, data quality.

10.3 Learning and Growth

- Translating business requirements into technical solutions
- Designing scalable systems that handle growth in data volume and complexity
- Problem-solving: Addressing real challenges like late data, schema evolution, and validation
- Documentation: Writing clear code, documenting design decisions, communicating results

10.4 Reflection on Internship Experience

This eight-week internship at Celebal Technologies provided an invaluable bridge between academic learning and industry practice. Working on the RetailStream project exposed me to modern data engineering practices, cloud-based analytics platforms, and the engineering mindset required in production systems.

10.5 Recommendations for Future Interns

- Focus on understanding architecture patterns before diving into specific tools.
- Practice modular code design and reusability from the start—it saves immense effort later.
- Prioritize data quality and validation; it's the foundation of reliable analytics.
- Document everything—your future self and teammates will thank you.
- Don't rush to production; prototype, validate, iterate, then deploy.

10.6 Final Remarks

The RetailStream Inc. pipeline is more than a portfolio project—it's a demonstration of End-to-End Data Engineering Pipeline Industry-Oriented Data Engineering Project applied to a realistic business scenario. The pipeline successfully handles four distinct data ingestion patterns, implements modern architectural patterns, provides comprehensive validation, and generates business-ready analytics.

I am confident that the skills, knowledge, and experience gained through this project will serve as a strong foundation for a career in data engineering. The internship at Celebal Technologies has equipped me not just with technical tools, but with an engineering mindset thinking in systems, prioritizing reliability, and always considering the downstream impact of decisions.

This concludes my Final Project Report for the Celebal Technologies Data Engineering Internship.

Submitted with confidence and professional pride,

[Sanjana]

Data Engineering Intern

Celebal Technologies

July 2026